

Otten - BI Engineer Role Technical Test

Dataset	Brazilian E-Commerce (Public PostgreSQL)
Submission	See instructions per test below

Submission instructions are detailed per-test on the following pages.

For any inquiries, kindly contact us via email rizki.mahardika@ottencoffee.co.id.

Submission Guidelines

Please read the submission requirements for each test carefully before you begin. All outputs must be clearly labelled and submitted together unless stated otherwise.

Test	Deliverables	Format
Test 1 Advanced SQL	SQL script file + written explanation	.sql + .pdf
Test 2 Dashboard	Dashboard screenshot + written explanation	.png/.jpg + .pdf
Test 3 Data Transformation Pipeline	SQL script file(s) in ZIP + written explanation	.zip + .pdf

1. Advanced SQL & Data Exploration

This test evaluates your ability to write clean, efficient, and well-structured SQL queries against a real PostgreSQL dataset. You will be given read-only access to a public PostgreSQL instance containing the Brazilian E-Commerce dataset.

Database information (choose one of these two):

1.

Host: aws-1-ap-northeast-1.pooler.supabase.com

Database: postgres

Port: 5432

User: public_readonly.hkmqxnppvspaoldrzzam

Password : moajj_masoa_javan

2.

Host: aws-1-ap-northeast-1.pooler.supabase.com

Database: postgres

Port: 5432

User: public_readonly.utrxerkurjdqflorzizo

Password : wanojakamojang2026#

Dataset Overview

- Source: Brazilian E-Commerce public dataset (Olist)
- Access: Public PostgreSQL instance
- Key tables: orders, order_items, order_payments, order_reviews, customers, products, sellers, geolocation, product_category_name_translation

Part A — Core SQL Queries

Write clean and well-commented SQL queries to answer each of the following questions. Each query should be executable as-is against the provided database.

A1	Monthly Revenue Trend
	Calculate the total revenue (sum of payment_value) per month, ordered chronologically. Include the month-over-month percentage change in revenue. Display: year, month, total_revenue, prev_month_revenue, mom_change_pct.

A2	Top 10 Product Categories by Revenue Identify the top 10 product categories (in English) ranked by total revenue. Include the number of distinct orders, total items sold, and average order value per category. Exclude cancelled orders.
A3	Customer Cohort Retention Group customers by their first purchase month (acquisition cohort). For each cohort, calculate how many customers made at least one additional purchase in the following 1, 2, and 3 months. Display cohort_month, cohort_size, retained_m1, retained_m2, retained_m3.

Part B — Query Optimisation

The following query runs slowly on large data. Refactor it to improve performance without changing the output. Briefly explain (in the .pdf) what changes you made and why.

```

SELECT
    revenue_rank,
    category,
    total_orders,
    total_revenue,
    ROUND((total_revenue * 100.0 / NULLIF(SUM(total_revenue) OVER (), 0))::numeric,
2)          AS revenue_share_pct,
    ROUND((SUM(total_revenue) OVER (
        ORDER BY total_revenue DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) * 100.0 / NULLIF(SUM(total_revenue) OVER (), 0))::numeric, 2)
AS cumulative_revenue_pct,
    avg_order_value,
    credit_card_revenue,
    boleto_revenue,
    credit_card_share_pct,
    late_count,
    late_rate_pct,
    avg_days_late,
    RANK() OVER (ORDER BY late_rate_pct DESC)
AS late_rank

```

```

FROM (
    SELECT
        RANK() OVER (ORDER BY SUM(b.price) DESC)
    AS revenue_rank,
        b.product_category_name_english
    AS category,
        COUNT(DISTINCT b.order_id)
    AS total_orders,
        SUM(b.price)
    AS total_revenue,
        ROUND((SUM(b.price) / NULLIF(COUNT(DISTINCT b.order_id), 0))::numeric, 2)
    AS avg_order_value,
        ps.credit_card_revenue,
        ps.boleto_revenue,
        ROUND((ps.credit_card_revenue * 100.0 /
            NULLIF(ps.credit_card_revenue + ps.boleto_revenue, 0))::numeric, 1)
    AS credit_card_share_pct,
        ls.late_count,
        ls.avg_days_late,
        ROUND((ls.late_count * 100.0 /
            NULLIF(COUNT(DISTINCT b.order_id), 0))::numeric, 1)
    AS late_rate_pct
    FROM (
        SELECT
            p.product_category_name,
            t.product_category_name_english,
            oi.order_id,
            oi.price
        FROM products p
        JOIN product_category_name_translation t
            ON t.product_category_name = p.product_category_name
        JOIN order_items oi
            ON oi.product_id = p.product_id
    ) b
    JOIN (
        SELECT
            p.product_category_name,
            SUM(pay.payment_value) FILTER (WHERE pay.payment_type = 'credit_card')
        AS credit_card_revenue,

```

```

SUM(pay.payment_value) FILTER (WHERE pay.payment_type = 'boleto')
AS boleto_revenue
    FROM products p
    JOIN order_items oi ON oi.product_id = p.product_id
    JOIN order_payments pay ON pay.order_id = oi.order_id
    GROUP BY p.product_category_name
) ps ON ps.product_category_name = b.product_category_name
JOIN (
    SELECT
        p.product_category_name,
        COUNT(DISTINCT CASE WHEN o.order_delivered_customer_date
            > o.order_estimated_delivery_date
            THEN o.order_id END)
AS late_count,
        ROUND(AVG(CASE WHEN o.order_delivered_customer_date
            > o.order_estimated_delivery_date
            THEN EXTRACT(EPOCH FROM (o.order_delivered_customer_date
                - o.order_estimated_delivery_date)) / 86400.0
            END)::numeric, 1)
AS avg_days_late
    FROM products p
    JOIN order_items oi ON oi.product_id = p.product_id
    JOIN orders o ON o.order_id = oi.order_id
    WHERE o.order_delivered_customer_date IS NOT NULL
    GROUP BY p.product_category_name
) ls ON ls.product_category_name = b.product_category_name
GROUP BY
    b.product_category_name_english,
    ps.credit_card_revenue,
    ps.boleto_revenue,
    ls.late_count,
    ls.avg_days_late
) ranked
ORDER BY revenue_rank;

```

In your explanation, address: (1) which specific bottlenecks you identified, (2) what SQL features or indexing strategies you applied, and (3) how you would verify the improvement in a real environment.

Deliverables

.sql file	A single well-organised SQL file containing all queries (A1–A3) and the optimised B query. Use comments to clearly label each section.
.pdf file	Written explanation covering: (1) your query optimisation approach for Part B, (2) any assumptions made, and (3) key insights you observed from the query results. Screenshots of query outputs are welcome.

2. Business Intelligence Dashboard

This test evaluates your ability to transform raw data into a coherent, stakeholder-ready BI dashboard. You will use Apache Superset connected to the same Brazilian E-Commerce PostgreSQL instance.

Setup

- Tool: Self-hosted Apache Superset/Other Tools Visualization (Docker recommended — see official docs)
- Data source: Brazilian E-Commerce public PostgreSQL instance (same credentials as Test 1)
- You may create new datasets using the SQL Lab feature (or similar) to build derived or joined tables

Dashboard Requirements

Build a single cohesive dashboard titled **"E-Commerce Business Overview"**. The dashboard must contain **at least 6 charts**, and designated to be used for a **Senior Manager** level user in the industry. Each chart must have a clear title and appropriate axis labels.

★ *Bonus Points:* 1. Add a Month-over-Month (MoM) revenue growth KPI card showing the percentage change compared to the previous month. 2. Add cross-filtering so that clicking one chart filters the others on the same dashboard. 3. Add at least one additional custom SQL dataset using a JOIN across 3 or more tables.

Tips & Notes

- Filters and date pickers at the dashboard level are encouraged
- You will present and walk through the dashboard during the interview phase
- Focus on **insight value** — a well-chosen chart with good labelling beats a complex one that is hard to read and no valuable insight for designated user of your dashboard

Deliverables

.png / .jpg	Full dashboard screenshot.
.pdf file	Written explanation for each chart: what it shows, why you chose that chart type, and the key business insight it conveys. Include chart screenshots if helpful.

3. Analytics Engineering with dbt & Pipeline Design

This test evaluates your ability to build a structured, production-grade analytics engineering layer using dbt (or similar), and to design a complete ELT pipeline that could support an evolving data warehouse in a real business environment.

Required Models

Build a data transformation project connected to the Brazilian E-Commerce PostgreSQL database.

Your project must follow the **staging** → **intermediate** → **mart** or **bronze** → **silver** → **gold** layered architecture.

Staging Layer — (stg_*)

Please choose one or more of the following examples:

- stg_orders — rename the columns to appropriate one, handle nulls, and cast data types appropriately
- stg_order_items — rename the columns to appropriate one, handle nulls, and cast data types appropriately
- stg_customers — rename the columns to appropriate one, handle nulls, and cast data types appropriately
- stg_products — rename the columns to appropriate one, handle nulls, and cast data types appropriately, join with category translation to produce English category names
- stg_order_reviews — rename the columns to appropriate one, handle nulls, and cast data types appropriately, clean review data, flag low scores (score <= 2) as a boolean

Intermediate Layer — (int_*)

Please choose one or more of the following examples:

- int_orders_enriched — join stg_orders with stg_customers, stg_order_items, and stg_order_reviews; calculate delivery_delay_days (actual minus estimated delivery); flag late_delivery as boolean
- int_seller_metrics — aggregate per-seller: total revenue, total orders, avg review score, late delivery rate

* You may enrich the intermediate layers by exploring other business metrics/feature engineering that you might think give valuable insights on the downstream table.

Mart Layer — (mart_*)

Please choose one or more of the following examples:

- mart_monthly_revenue — monthly aggregation of revenue, order count, and MoM growth rate
- mart_customer_segments — segment customers into 'new', 'returning', and 'churned' based on purchase history (define your own business logic and document it)
- mart_product_performance — top products and categories ranked by revenue, units sold, and review score

* You can explore and determine your own pipelines that you find interesting and provides business insights aside from the examples (Staging - Int - Mart)

Your design must address:

- **Loading:** How would you load raw data into the warehouse (full overwrite, incremental deduped, incremental append, etc) ? How would you handle schema changes in the source?
- **Orchestration:** Which tool would you use to schedule and monitor the pipeline (e.g., Airflow, Prefect, Cloud Composer, dbt Cloud)? Justify your choice.
- **Data Quality:** What checks would you implement to ensure data reliability at each stage?

Tips & Notes

- You will present and walk through the design during the interview phase

Deliverables

.zip file	Must include: One .sql file for each staging → intermediate → mart layer. Use comments to clearly label each transformation model.
.pdf file	Written explanation covering: (1) your model design decisions and layering rationale, (2) your choice of orchestration tool and justification. Screenshots and simple architecture/workflow diagrams are strongly encouraged.